Guided Tutorial on Search (BFS, DFS, A*)

This guided tutorial is designed to help you understand how search algorithms work in Artificial Intelligence, particularly Breadth-First Search (BFS), Depth-First Search (DFS), and A* Search. We will use the **Romania map problem** from the AIMA textbook to illustrate these concepts clearly with diagrams and simple step-by-step reasoning.

---

1. Key Definitions (In Simple Terms)

Term	Meaning
State	A representation of where you are in the problem (e.g., being in the city of Arad).
State Space	All the possible states you can be in (all cities and connections in the Romania map).
Action	A possible move (e.g., drive from Arad to Sibiu).
Transition Model	Rules that say what state you reach after an action.
Search Node	A data structure that stores a state along with path and cost information.
Search Tree	A tree that shows how states are expanded during search.
Goal	The target state (e.g., reaching Bucharest).
Branching Factor	The number of immediate neighbors from any state.

Romania Map

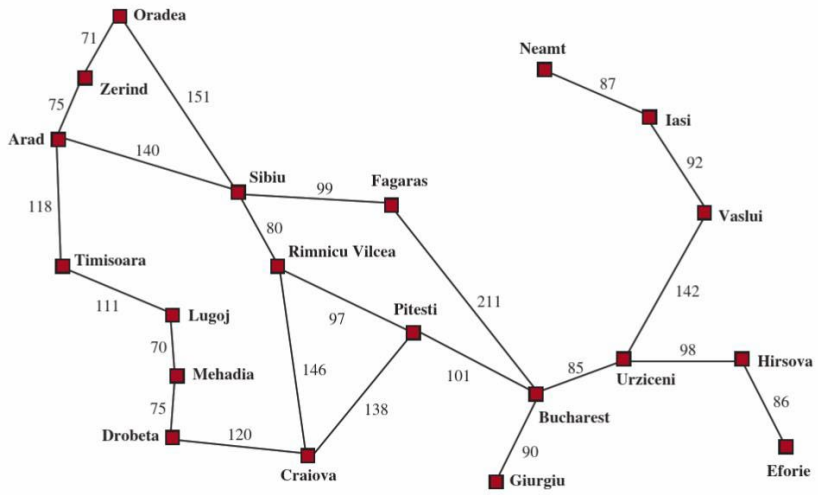


Figure 3.1 A simplified road map of part of Romania, with road distances in miles.

2. Breadth-First Search (BFS)

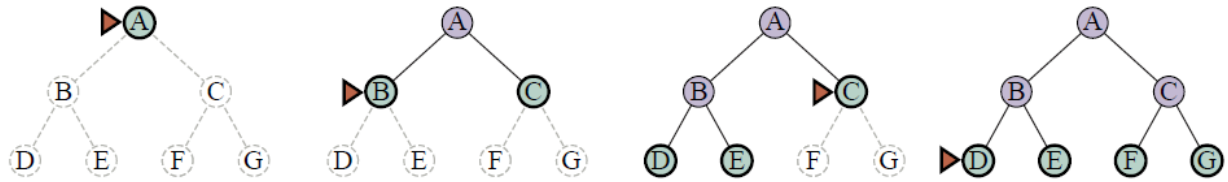


Figure 3.8 Breadth-first search on a simple binary tree. At each stage, the node to be expanded next is indicated by the triangular marker.

BFS explores the *shallowest* nodes first — level by level.

BFS always finds the *shortest path in terms of the number of steps*.

3. Depth-First Search (DFS)

DFS goes as deep as possible before backtracking.

Example (may take a longer or incorrect path first)

```
Arad
 → Sibiu
  → Rimnicu Vilcea
   → Craiova
    → Drobeta
     → Mehadia
      → Lugoj
       → Timisoara (dead end, backtrack)
```

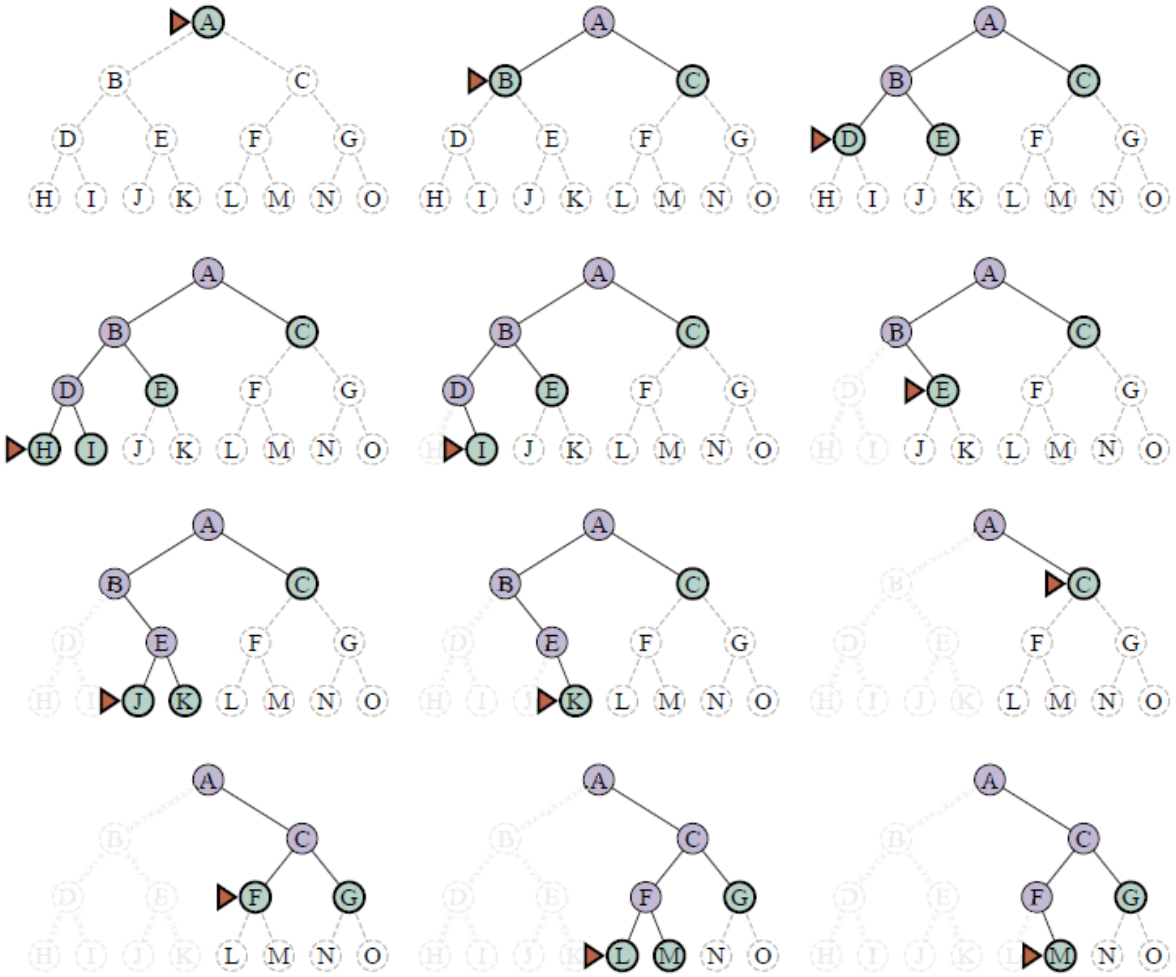


Figure 3.11 A dozen steps (left to right, top to bottom) in the progress of a depth-first search on a binary tree from start state A to goal M. The frontier is in green, with a triangle marking the node to be expanded next. Previously expanded nodes are lavender, and potential future nodes have faint dashed lines. Expanded nodes with no descendants in the frontier (very faint lines) can be discarded.

DFS does not guarantee the shortest path.

---

4. A* Search (Heuristic + Cost)

A* uses the formula:

`f(n) = g(n) + h(n)`

Where: - **g(n)** = cost from start to node n - **h(n)** = heuristic (straight-line estimate to goal)

Example: Starting from **Arad**

City	g(n)	h(n)	**f(n) = g+h**
Sibiu	140	253	**393**
Timisoara	118	329	**447**
Zerind	75	374	**449**

A* selects **Sibiu** because it has the lowest f(n).

Continuing from Sibiu:

City	g(n)	h(n)	**f(n) = g+h**
Fagaras	140+99=239	176	415
Rimnicu Vilcea	140+80=220	193	413

A* will choose Rimnicu Vilcea next.

---

5. Comparison of Algorithms

Feature	BFS	DFS	A*
Uses Heuristic	**No**	**No**	**Yes**
Guarantees Shortest Path	**Yes**	**No**	**Yes**
Memory Usage	High	Low	Moderate
Completeness	**Yes**	**No** (may go infinite)	**Yes**

---

6. Summary

- Use **BFS** when you want the shortest path and memory is not a limitation.
- Use **DFS** when memory is limited and the solution is deep.
- Use **A*** when you have a good heuristic and want an optimal and efficient path.

---

7. Try It Yourself (Exercises)

1. Using BFS, find a path from **Timisoara** to **Bucharest**.
2. Using DFS, show the first 7 nodes expanded from **Oradea**.
3. Using A*, calculate f(n) values starting from **Sibiu**.
